BFS and DFS in interview shape

Week 4, Lecture 2 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026 · Senior CS students

What we'll cover today

- **BFS on trees and grids:** the queue template, level order, shortest path
- **DFS via recursion and stack:** pre-order, in-order, post-order
- **Graph modeling:** turning a real problem into nodes and edges
- **Backtracking:** constrained DFS with a decision tree and pruning

BFS on trees and grids

BFS processes nodes level by level

BFS uses a queue. It visits every node at depth d before any node at depth d+1.

This property gives BFS one guarantee DFS never has:

On an unweighted graph, the first time BFS reaches a node, it has found the shortest path to that node.

This is why BFS answers "shortest path" questions. DFS does not.

The BFS template

```
from collections import deque

def bfs(root):
    if root is None:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        # process node
        for neighbor in node.neighbors:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Mark nodes visited **when you enqueue them**, not when you dequeue.

Marking on dequeue allows duplicates in the queue. Marking on enqueue prevents them.

BFS on a grid

Model each grid cell as a node; cardinal-direction neighbors are edges.

The BFS template from the previous slide applies without modification.

- Initialize the queue with all starting cells (multi-source BFS)
- Mark each starting cell visited on enqueue
- Expand neighbors in the four cardinal directions
- Track the step count as a third element in each queue entry

This pattern covers Rotting Oranges (LC 994), Walls and Gates (LC 286), and Pacific Atlantic Water Flow (LC 417).

DFS via recursion and stack

DFS explores one path fully before backtracking

DFS uses a stack: either the call stack (recursion) or an explicit stack (iteration).

Three orderings for binary trees:

- **Pre-order:** process node, then left, then right
- **In-order:** left, then node, then right (gives sorted output on a BST)
- **Post-order:** left, then right, then node (useful when children must resolve before parent)

In-order traversal: recursive vs. iterative

```
# Recursive
def inorder(root):
    if root is None:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Iterative (explicit stack)
def inorder_iter(root):
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result
```

A tree is a special case of a graph

Back To Back SWE (2019): a tree is a graph with no cycles and one designated root.

The DFS you know on trees works on graphs with one addition:

```
def dfs(node, visited):
    if node in visited:
        return
    visited.add(node)
    # process node
    for neighbor in graph[node]:
        dfs(neighbor, visited)
```

The visited set is what prevents infinite loops on graphs with cycles.

Trees do not need it. Graphs do.

Graph modeling

Graph modeling: three questions first

Before writing any traversal:

1. What are the nodes?
2. What are the edges?
3. What question am I answering?

Question 3 picks the traversal.

Use BFS for:

- Shortest path (unweighted)
- Level order, "minimum steps"
- Multi-source spreading

Use DFS for:

Backtracking as constrained DFS

Backtracking is DFS with a decision tree

At each step you make a choice, recurse on that choice, then undo it.

```
def backtrack(candidates, start, path, result, target):
    if sum(path) == target:
        result.append(path[:])
        return
    if sum(path) > target:
        return              # pruning: no need to go deeper
    for i in range(start, len(candidates)):
        path.append(candidates[i])
        backtrack(candidates, i, path, result, target)
        path.pop()          # undo the choice
```

The `path.pop()` is what makes it backtracking rather than plain DFS.

Pruning cuts the search space

An unpruned backtrack explores every leaf of the decision tree: O(n!) in the worst case.

Pruning returns early when a partial path cannot lead to a valid solution.

- Sum exceeds target: prune
- Remaining candidates cannot fill the required count: prune
- Duplicate element at the same level of the tree: prune with a seen set

State your pruning conditions before writing the recursion. Interviewers award credit for reasoning about the search space.

Take-aways

1. BFS uses a queue and guarantees shortest path on unweighted graphs. Mark nodes visited on enqueue, not dequeue.
2. DFS uses the call stack or an explicit stack. In-order traversal of a BST gives sorted output; post-order resolves children before parent.
3. A tree is a graph with no cycles. DFS and BFS on trees work on general graphs by adding a visited set.
4. Graph modeling comes before code: name the nodes, name the edges, name the question. The question determines the traversal.
5. Backtracking is DFS plus an undo step. State pruning conditions before writing the loop.

> "A tree is just a graph with no cycles and one designated root. The BFS you already know on trees works on graphs with a visited set added."

Back To Back SWE, Binary Tree Level Order Traversal (2019)

What's next

- **Section (this week):** Pattern sprint 2, three timed mediums
- **Reading:** Week 4 reading, recursion, trees, and graph traversal
- **Week 5:** Dynamic programming and interview communication
- **NeetCode roadmap:** Graphs and Backtracking sections for practice